Governance-Driven Agentic Coding

Give agents bounded authority. Require evidence for claims. Separate production from review. Keep acceptance human.

> **Owner review candidate**
>
> This is an unpublished v1.5 candidate, not the current public method. Frozen v1.4 HTML and PDF artifacts are unchanged.
>
>

The problem it addresses

Agents can produce implementation quickly. Speed does not create authority, correctness, or accountability. GDAC is a small delegation protocol that states who may do what, what counts as evidence, who can block, and when work must stop.

> **It is not a full agent runtime or a compliance certification.**
>
> It does not replace Codex, Claude, CI, code review, or organizational governance. It connects a verifiable outcome, bounded execution, evidence, and an accountable Owner decision.

The construction can be reused across software domains because its core fields are not tied to one business domain. The evidence is not domain-neutral. It comes from two author-run private projects that external readers cannot inspect, and it has not been independently replicated.

Only three controls carry the method

Bounded delegation

Freeze the verifiable result, non-goals, write scope, budget, stop conditions, and Owner-reserved decisions before work begins. An agent cannot expand its own authority.

Context-independent review

A reviewer who did not inherit the Builder's reasoning rechecks the governed object. This reduces shared blind spots. It is not organizational independence, third-party validation, or internal audit.

Claim integrity

Facts, inferences, assumptions, unknowns, and Owner decisions remain distinct. Existing code does not prove execution. Passing tests do not prove acceptance. Matching hashes do not prove semantic correctness.

The governed operating loop

1 **Freeze the outcome**
Write verifiable acceptance criteria before implementation.

2 **Bound authority**
Declare writable objects, forbidden actions, budgets, retry limits, and stop conditions.

3 **Build the minimum change**
Reuse the existing architecture and implement only what the frozen criteria require.

4 **Produce evidence**
Prefer deterministic checks, then add rule, model, human, or Owner evaluation in proportion to risk.

5 **Challenge independently**
Review the frozen object and original records for omissions, overreach, invalid inference, and insufficient evidence.

6 **Accept or stop**
Technical verification supports an Owner decision. It does not make that decision.

> The contract-to-evidence chain is the skeleton, not the product claim. The essential rule is that every state change needs declared authority and sufficient evidence. Missing evidence cannot silently advance the work.

The Outcome Contract is executable

Use the Contract for material or delegated work. Shrink it for low-risk work instead of completing fields ceremonially.

Field	Failure it prevents
Outcome and acceptance criteria	Treating activity as an accepted result.
Non-goals	Scope expanding during execution.
Authority	An agent changing scope, tests, budget, or publication rights.
Budget and stop conditions	Unbounded retries, recursion, and sunk-cost continuation.
Evidence requirements	Substituting easy evidence for evidence the claim requires.
Handoff	Delivering output without failures, limits, and unresolved decisions.

```
python -m pip install -r requirements-dev.txt
python practice-kit/tools/validate_contract.py \
  practice-kit/examples/outcome-contract.example.yaml
```

The repository includes a machine-readable JSON Schema, a filled example, and the validator.

Authority and review

The method needs three responsibilities. A complex project may add an Orchestrator, Verifier, Red Team, or Integrator, but extra roles do not change Owner-reserved authority.

Responsibility	May do	May not do
Owner	Freeze outcomes and authority, accept work, and decide release, risk, and material method changes.	Treat missing evidence as a pass or transfer accountability to an agent.
Producer	Implement within declared write scope and budget, recording tests, failures, and limits.	Expand authority, weaken acceptance criteria, or declare its own work accepted.
Reviewer	Rejudge the frozen object from original evidence and block unsupported claims.	Present context separation as third-party independence or replace Owner acceptance.

Minimum review separation

- The Reviewer does not inherit the Producer's hidden reasoning or conclusion summary.
- Inputs are the frozen object, acceptance criteria, diff, tests, and original records.
- The Producer cannot unilaterally dismiss material findings.
- The same model may perform separate roles, but this provides context separation, not party independence.

Evidence precedes claims

Evidence states

Fact: directly reproducible from a record.

Inference: supported by facts but still interpretive.

Assumption: an explanation or prediction not sufficiently tested.

Unknown: the available material cannot support judgment.

Owner decision: acceptance, risk appetite, or authorization.

Three claim rules

Capability Claim Attestation: `agent-ready` is not `agent-executed`. This is a named rule that can be applied and failed on its own.

Conclusion Provenance: a conclusion traces to its object, version, evidence, and decision authority.

Derivation Integrity: a hash proves object identity, not semantic correctness from source to conclusion.

Deterministic tests come first. Model graders can supplement capability or adversarial evaluation, but their configuration, inputs, outputs, and failures must be recorded. If the claim lacks its required evidence, the state is blocked or unknown.

Learn from failure without self-authorization

A project may propose a method change. It cannot activate a global rule. Candidate rules move through `candidate`, `validated`, `active`, and `retired`, with tests, logs, diffs, user corrections, or preventive checks attached.

- Freeze the baseline, candidate delta, and decision rule before seeing the outcome.
- Ask whether the candidate changes a material decision or detects a material error earlier.
- Narrow or reject changes that add fields, duplicate existing controls, or create disproportionate rework.
- At closeout, record retained assets, prohibited claims, and unresolved unknowns instead of continuing because of sunk cost.

Copy the reduced Method-Change Re-test and Project Closeout directly.

Minimum adoption

Do not start by creating an AI organization. Select one valuable, reversible, testable task.

1. Copy and reduce the Outcome Contract, freeze it, and run the validator.
2. Give the Producer only necessary write access and one or two retries.
3. Give the Reviewer read-only access to the frozen object, diff, tests, and original evidence.
4. Have the Owner record accepted, rework, blocked, or stopped.
5. Measure only when instrumentation is real: tokens per accepted outcome and the share of accepted outcomes reopened because original evidence was insufficient.

Without usage evidence, do not report improved efficiency, quality, or risk reduction.

Current evidence and limits

> **Evidence independence statement**
>
> Unless a specific record says otherwise, current GDAC documents, examples, tests, reviews, and conclusions were produced, recorded, and evaluated by the author with AI assistance. No independent third party has replicated the method or validated improved delivery, safety, cost, or governance outcomes.

- v1.5 is an Owner review candidate, not a published release.
- A reusable structure is not evidence of cross-domain validation.
- An AI Act or regulated-environment overlay can map evidence and flag risk. It cannot certify compliance.
- Current tests show that the schema and validator work on the stated cases. They do not prove organizational value.

See Related Work and Boundaries for the relationship to NIST AI RMF, the IIA Three Lines Model, SR 26-2, the EU AI Act, ADRs, in-toto, SLSA, preregistration, Inspect, and AGENTS.md.